EXPENSE TRACKER · ENGINEERING GUIDE

FULL-STACK CODE TOUR

How the app works

A practical explanation of the project's architecture, business logic, data flow, database rules, and the TypeScript, React, Python, and SQL syntax used throughout.

Project: Expense Tracker MVP
Stack: React 19 · TypeScript 6 · Vite · FastAPI · Supabase Auth/Postgres
Guide scope: application source, supporting modules, tests, configuration, schema, and migrations

01 · ORIENTATION

What this codebase does

The project is a browser-based finance tracker with two operating modes: local fictional demo data, or an authenticated Supabase-backed application.

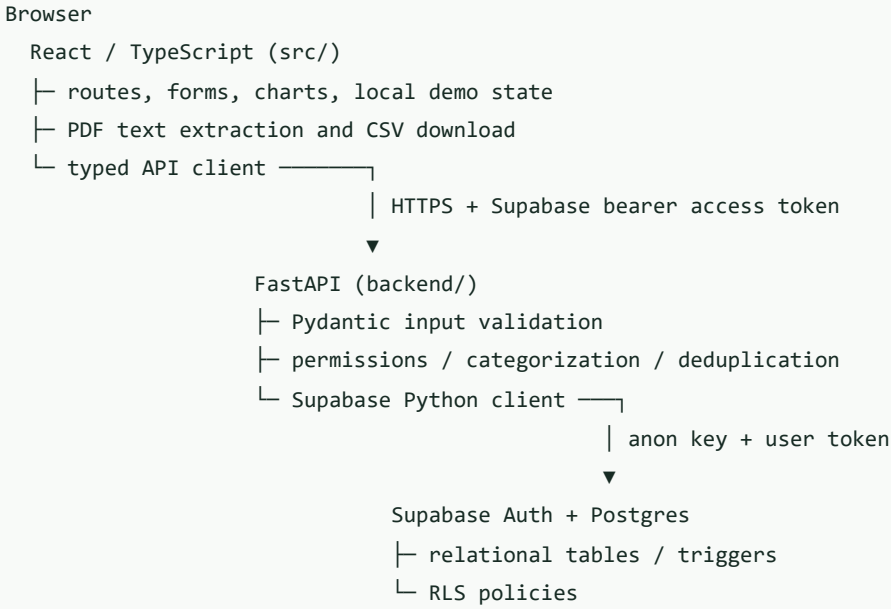
Core product features

Personal and shared organization workspaces; income and expense transactions; monthly budgets; summaries and category charts; search and CSV export; PDF bank-statement import with a human review step; member and role administration; support conversations.

Responsibility boundaries

React renders screens and computes display summaries. FastAPI validates requests and applies business rules. Supabase Auth identifies users, Postgres stores data, and row-level security (RLS) enforces the final data-access boundary.

Architecture at a glance



> **Trust boundary:** A selected workspace in the UI is not proof of permission. The API validates identity, database requests carry the user's token, and Postgres RLS policies decide whether each row can be read or changed.

End-to-end request example

1. A user submits a transaction form in `src/App.tsx`.
2. The UI converts form strings to typed data (for example, `Number(amount)`) and invokes `createTransaction()` in `src/lib/api.ts`.
3. The API client attaches the current Supabase access token and JSON body to `POST /api/transactions`.
4. FastAPI validates the payload against `TransactionInput`, gets the verified user from a dependency, and writes through Supabase.
5. Postgres constraints, triggers, and RLS validate ownership/category scope; the returned database row is converted back into the frontend `Transaction` shape.

02 · FRONTEND STRUCTURE

React application and navigation

`src/main.tsx` is the browser entry point. It imports global CSS, finds the HTML element with id `root`, and renders `<App />` inside React `StrictMode`. The null assertion (`!`) tells TypeScript the root element exists in `index.html`.

`src/App.tsx` is the application coordinator. `App` provides `BrowserRouter`; `AppContent` resolves sign-in, password recovery, demo access, and the public landing page; `ExpenseApp` owns workspace data, screen state, handlers, and authenticated routes.

Routes and user-visible screens

Path	Screen / behavior	Primary implementation
`/`	Dashboard: date presets, totals, income/expense trend, category breakdown, latest transactions.	`App.tsx` + `utils/finance.ts`
`/transactions`	Search/filter table, category correction, add/edit/delete, PDF import.	`App.tsx` + transactions feature
`/budgets`	Monthly budget cards, spent/remaining totals, progress bars.	`App.tsx` + finance utilities
`/reports`	Summary and category totals with CSV export.	`App.tsx`
`/organization`	Create a shared workspace and inspect current workspace roles.	`OrganizationPage.tsx`
`/admin`	Organization activity, invitations, member roles and removals; UI is conditionally shown to workspace admins.	`AdminDashboard.tsx` + backend admin router
`/support`, `/settings`, `/account`	Ticket conversations, profile preferences, and account/spending overview.	Support, App, and Account components
`/login`	Sign in, signup, email recovery, new password; query parameter `?mode=sign-up` selects signup.	`AuthPage.tsx`

The page shell uses React Router's `Routes`/`Route`, `Link` for navigation without a reload, and `NavLink` for active navigation styling. Admin navigation is hidden unless the selected organization role is owner/admin; the backend independently verifies that role.

Application state and effects

- `useState` stores active session, transactions, budgets, profile, workspaces, categories, selected workspace, filters, dialogs, and notices.
- `useEffect` subscribes to Supabase auth changes, bootstraps remote data, persists demo data to `localStorage`, performs uncategorized-transaction backfill, and expires notices. Cleanup functions unsubscribe, clear timers, or mark async results inactive.
- `useMemo` caches derived ranges, filtered transactions, summaries, trend data, and budget/category views until their dependencies change.
- Handlers use functional updates such as `setTransactions(current => ...)` so updates use the latest state snapshot.

> **Mode switch:** authenticated requests go through the remote API; demo mode uses the same screens but reads/writes fictional data in browser storage. Demo data is not a substitute for remote persistence and contains no live account information.

03 · TYPESCRIPT & REACT SYNTAX

How to read the frontend code

Types describe valid data

`src/types.ts` defines the shared data vocabulary. TypeScript's unions restrict values; interfaces describe object shapes; optional properties use `?`.

```
export type TransactionType = 'income' | 'expense'
export interface Transaction {
  id: string
  workspaceId: string
  categoryId: string
  type: TransactionType
  amount: number
  transactionDate: string
}
```

`Transaction['type']` looks up the type of one property. `Omit<Transaction, 'id' | 'createdAt'>` makes a draft shape that deliberately excludes server/generated values. `Record<string, string>` is a string-keyed dictionary. `Promise<T>` is the eventual result of asynchronous work.

JSX is UI expressed as JavaScript

```
<button onClick={() => setShowBudgetForm(true)}>
  Create budget
</button>
{budgets.map(budget => (
  <BudgetCard key={budget.id} budget={budget} />
))}
{budgets.length === 0 && <p>No budgets yet</p>}
```

Curly braces evaluate JavaScript inside markup. `map` turns records into repeated elements; stable `key` values help React reconcile lists. `&&` conditionally renders content. Props pass data and callbacks down into smaller components.

Forms, events, and async work

```
const save = async (event: FormEvent<HTMLFormElement>) => {
  event.preventDefault()
  const values = new FormData(event.currentTarget)
  const amount = Number(values.get('amount'))
  const saved = await createRemoteTransaction(workspaceId, draft)
  setTransactions(current => [saved, ...current])
}
```

The event type gives editor/type-checker support. `preventDefault()` stops browser navigation; `FormData` reads named controls; `await` pauses until the API resolves. Error handling uses `try/catch` and displays a user-facing notice. The `as` keyword is a type assertion; use it only when the UI element's allowed values have already been constrained.

Reusable components by feature

- `AuthPage.tsx`: one form switches between sign-in, signup, recovery, and new-password modes; Supabase Auth performs the identity operation.
- `AccountPage.tsx`: profile name editing, workspace role, overall income/expense summaries, category spending, six-month charts, recent activity, account actions.
- `OrganizationPage.tsx`: creation form and organization list; handles a busy state and reports errors.
- `AdminDashboard.tsx`: async load/refresh of organization overview and membership; invite, role change, remove operations; owner-specific controls.
- `SupportCenter.tsx`: tickets/messages refresh periodically; demo tickets are local, authenticated tickets use API methods; workspace admins can update status.
- `StatementImportDialog.tsx`: dynamically loaded on demand because PDF.js is comparatively large; shows extracted rows for editing before import.

04 · DOMAIN LOGIC

Financial calculations and categories

`src/utils/finance.ts` is a mostly pure module: it receives arrays/ids/dates and returns calculated values, making it straightforward to test.

Function	Logic	Important detail
`formatCurrency`	Uses `Intl.NumberFormat` with currency display and two decimal places.	Default currency is NGN; the active workspace currency is passed by screens.
`calculateSummary`	Filters by workspace and inclusive date range; sums income and expenses; net = income − expense.	Uses `date-fns` ISO parsing/date comparisons.
`buildTrendData`	Creates daily or monthly buckets, initializes missing dates to zero, adds amounts per transaction.	Dashboard selects daily for shorter presets and monthly for all history/12 months.
`buildCategoryBreakdown`	Includes expenses only; resolves category names; groups and sorts totals descending.	Missing category records display as "Uncategorized".
`buildBudgetStatus`	Totals matching workspace expense transactions from budget start through that month's end.	Optional category id scopes a budget; progress is guarded against non-finite values.
`filterTransactions`	Combines optional workspace, type, category, description search, and date bounds.	Every failed condition excludes the row; empty filters do not constrain it.
`canAccessWorkspace`	Personal workspace owner or organization membership determines access.	UI helper only; database RLS remains authoritative.

Category suggestion rules

`src/features/transactions/categorySuggestions.ts` contains ordered regular-expression rules. Each rule has a pattern, candidate category names, and transaction type. For example, salary words match income categories while transport/merchant words match expense categories. Matching is case-insensitive, and only category names that exist in the current workspace and support the transaction type are returned. Fallbacks are "Other income" or "Other expenses" (then workspace-specific alternatives); the function returns `null` if none exist.

`backend/categorization.py` mirrors the policy for server-side import/backfill. It also recognizes income language and can correct a legacy expense type before assigning a category. Backend categorization is required because clients can bypass browser logic.

Statement text parser

`statementParser.ts` transforms lines of selectable PDF text into draft records. Regexes find dates and currency-formatted amounts. A date normalizer handles ISO, numeric, and common month-name formats; each line is cleaned of date/amount/currency markers. Explicit plus/minus signs, parentheses, debit/credit abbreviations, and descriptive keywords infer income versus expense. Uncertain sign-less rows default to expense. Parsed descriptions are capped at 240 characters and category initially remains empty.

> **Limits:** PDF.js extracts text in the browser, grouping text tokens by vertical position then x-coordinate. The app limits files to 15 MB and does not OCR scanned/image-only statements. Parsing is heuristic, so every row is editable and reviewed before saving.

05 · STATEMENT IMPORT LIFECYCLE

From PDF to saved transaction

1. User chooses a PDF. The file is read with `arrayBuffer()`; PDF.js visits each page and extracts text. The PDF itself is not uploaded.
2. Extracted lines go to `parseStatementText()`. Unsupported or empty text yields an explicit error.
3. The dialog sends parsed drafts to preview. The backend checks category ownership/type, suggests a valid category, and checks existing fingerprints. Demo mode performs equivalent preview logic locally.
4. The user may edit date, description, type, amount, or category. Preview refreshes after edits; duplicate and auto-category status are shown.
5. On submit, only reviewed transaction data is sent to the API. Server-side category checks and deduplication are repeated before writing.
6. The result reports total, new, duplicate, categorized, and uncategorized counts.

Duplicate fingerprint algorithm

The backend builds a stable text key from scope + date + type + normalized amount + currency + normalized description, then hashes it with SHA-256. Scope is the organization id for organization workspaces, or user id for personal data. Description whitespace is collapsed, trimmed, and lowercased; amount is quantized to two decimals. The database has a unique index on non-null fingerprints. The preview identifies existing and repeated-within-file rows; the final upsert still protects against duplicates arriving concurrently.

```
parts = (scope, date, type, amount_to_2dp, currency, normalized_description)
fingerprint = sha256("|".join(parts).encode("utf-8")).hexdigest()
```

The migration backfills unique legacy rows where safe. Ambiguous duplicates are intentionally not assigned the same unique key during migration, avoiding a destructive guess about which old records to preserve.

06 · API CLIENT

Frontend ↔ backend contract

`src/lib/api.ts` centralizes HTTP calls. Its generic `request<T>()` obtains the active Supabase session, rejects missing/expired sessions, sets the Bearer authorization header and JSON content type, calls `fetch`, translates API error details into exceptions, handles HTTP 204, and parses JSON as the expected result type.

Backend snake_case records are mapped into frontend camelCase domain objects by `toTransaction`, `toBudget`, `toCategory`, and support mappers. This keeps naming conventions local to each side of the boundary.

API group	Representative routes	Purpose
Bootstrap/profile	`GET /api/bootstrap`, `GET/PATCH /api/profile`	Load user, default workspace, memberships, categories, transactions and budgets; update profile.
Transactions	`GET/POST /api/transactions`, `PATCH/DELETE /api/transactions/{id}`	List, create, edit, delete and categorize a transaction.
Import	`POST /api/transactions/import/preview`, `POST /api/transactions/import`	Validate, preview, categorize, deduplicate and save a batch.
Categories / budgets	`GET /api/categories`, `GET/POST /api/budgets`	Read workspace categories and manage monthly budgets.
Organizations / admin	`POST /api/organizations`; `/api/admin/overview`, `/api/admin/members`	Create organizations; inspect activity; invite, update or remove members.
Support	`/api/support/tickets`, `/api/support/tickets/{id}/messages`	List/open tickets, read/reply, and update status.
Health/debug	`GET /health`, `GET /api/debug/auth`	Public health probe and authenticated identity diagnostic.

In Vite development, the API base URL is empty so relative `/api` requests use the configured dev proxy. In production, `VITE_API_URL` may supply the API origin. Network failures and non-success responses are surfaced rather than silently treated as success.

FastAPI request processing

`backend/main.py` defines the core app and financial routes. Pydantic models describe allowed request bodies. `ConfigDict(extra="forbid")` rejects unknown fields. `Field` constraints enforce positive amounts, precision, maximum string sizes and uppercase three-letter currencies. `Literal` limits status/type values; `UUID` and `date` parse and validate identifiers/dates.

```
class TransactionInput(BaseModel):
    model_config = ConfigDict(extra="forbid")
    category_id: UUID | None = None
    transaction_type: Literal["income", "expense"]
    amount: Decimal = Field(gt=0, max_digits=14, decimal_places=2)
    currency: str = Field(pattern=r"^[A-Z]{3}$", default="NGN")
    transaction_date: date
```

Python's `|` means a union/optional type, `dict[str, ...]` and `list[...]` declare containers, and `->` gives a function return type. Decorators such as `@app.post(...)` register HTTP routes. FastAPI injects `Auth` dependencies, parses query parameters, validates request models, and turns returned Python values into JSON responses.

Core server helpers

- `workspace_filter()` handles the special personal identifier or validates an organization UUID and adds the corresponding database filter.
- `transaction_values()` stamps trusted user/workspace ids from the authenticated context rather than accepting those security fields from the client.
- `_workspace_categories()` loads only categories visible to that workspace; `_validate_import_categories()` rejects unknown, cross-workspace, or wrong-type category ids.
- `_raise_import_database_error()` maps known database failures to explicit HTTP statuses and useful migration/permission errors; unknown failures are logged.
- Import batches are limited to 1–500 rows. Money uses `Decimal` rather than binary floating point in server validation and fingerprinting.

Security dependency and role checks

`backend/security.py` reads the Authorization Bearer token, validates it through Supabase Auth, then configures PostgREST to use the same token. The dependency returns an `AuthenticatedClient` containing client, verified user id, email, and metadata. Missing/invalid credentials produce 401; missing Supabase server settings produce 503. `require_workspace_admin()` checks membership role; `require_platform_admin()` checks the staff table. The server-side service-role client is isolated to invitations in `backend/admin.py` and is never part of browser variables.

Permissions and specialized routes

Organization member management

`backend/admin.py` is an `APIRouter` mounted under `/api/admin`. Pydantic models restrict invite email/role and role-change values. Owners and admins may manage a workspace, but only owners can invite/promote admins. A requester cannot remove or change their own role; owner membership cannot be demoted/removed; admins cannot manage another admin.

New users are invited using the backend-only service-role key. If the email already belongs to an account, an authenticated database RPC adds that user directly. Database errors are mapped to conflict/not-found/service errors; a failure after sending an invitation is explicitly reported because email delivery and membership insertion are separate operations.

Support tickets

`backend/support.py` defines ticket/message/status payloads and routes. Subjects have 4–140 characters; messages 1–3000; statuses are only open, in-progress, or resolved. Ticket list/create scopes to personal or a UUID organization. Status updates load the ticket first, then authorize an organization admin or platform support admin according to ticket scope. Ticket message access is additionally protected by RLS.

CORS and local development

FastAPI CORS origins are read from `CORS_ORIGINS`, defaulting to `http://localhost:5173`. Methods are limited to GET, POST, PATCH and DELETE; only authorization/content-type headers are accepted. `vite.config.ts` proxies `/api` and `/health` to local port 8000.

`scripts/dev.mjs` starts Uvicorn and Vite as child processes, preferring a project virtualenv Python when available, forwarding process output, and stopping its peer if either process exits. It handles SIGINT/SIGTERM for orderly shutdown.

09 · DATABASE

Postgres schema and access control

`supabase/schema.sql` creates the relational model. A UUID primary key identifies rows; foreign keys connect records; `check` constraints protect valid values; unique indexes prevent duplicates; `timestamptz` records instants. `numeric(14,2)` is used for money.

Table	What it stores / key relationships
`profiles`	Auth-linked user name and preferred currency; id references `auth.users`.
`organizations`	Shared workspace name/currency/creator.
`organization_members`	Organization-user join table; roles owner/admin/member; unique membership per pair.
`categories`	Personal or organization categories; a check requires exactly one owner scope.
`transactions`	User/workspace, optional category, income/expense, positive amount, currency/date, optional fingerprint.
`budgets`	User/workspace, optional category, positive monthly limit and start date.
`platform_admins`	Explicit server-side support staff allowlist.
`support_tickets`, `support_messages`	Ticket ownership/scope/status and ordered conversation messages.

Triggers and helper functions

- On auth-user creation: create profile and default personal categories; optionally create an organization when signup metadata requests one.
- On organization creation: create the owner's membership and default organization categories.
- On support message insert: update ticket timestamp so ticket lists sort by recent activity.
- Before transaction/budget category changes: verify the category belongs to the same personal user or organization. Invalid cross-scope assignments raise an exception.
- Stable security-definer helpers check organization membership/role and platform-admin membership for policy expressions.

Row-level security (RLS)

RLS is enabled on all application tables. Policies encode per-table actions: users read/update their own profile; organization members view workspace data; creators and/or owners/admins can update/delete organization transactions; organization budget/category management is admin-scoped; support ticket participants/admins see messages. A server/API role check improves error handling, but RLS is the database authorization boundary.

> **SQL syntax:** `CREATE POLICY ... USING (...)` decides which existing rows a statement may target; `WITH CHECK (...)` decides whether inserted/updated row values are allowed. `auth.uid()` is the current Supabase user identity.

Schema evolution files

The base schema describes a fresh database. Files in `supabase/migrations/` are incremental SQL updates for existing projects; apply the ones required by the deployed schema, as the README instructs. Do not assume a migration ran merely because its file exists.

Migration	Purpose
`20261008110000_transaction_import_fingerprint.sql`	Adds transaction fingerprint and unique index; safely backfills unambiguous legacy transactions for import deduplication.
`20261008173000_debug_auth_context.sql`	Defines an authenticated diagnostic RPC for comparing backend identity with database `auth.uid()`.
`20261009130000_fix_organization_insert_policy.sql`	Repairs the authenticated organization-insert RLS policy for creation.
`20261009131000_create_organization_rpc.sql`	Adds a security-definer create function that sets creator from trusted `auth.uid()`; grants execution to authenticated users only.
`20261009140000_admin_member_management.sql`	Adds protected functions to list members and add an existing user by email with owner/admin rules; restricts function grants.
`20261009160000_seed_existing_workspace_categories.sql`	Fills missing default categories in existing personal and organization workspaces without duplicating names.
`20261009161500_add_other_categories.sql`	Adds “Other expenses” and “Other income” fallbacks to existing workspaces if missing.

`supabase/seed_default_categories.sql` is a separate idempotent-style seed script for existing personal and organization categories. The initial schema also seeds personal defaults and has a creation trigger for organization defaults.

How behavior is verified

Frontend tests (Vitest)

- `src/utils/finance.test.ts`: formatting, summaries, date/workspace filtering, trend buckets, category breakdown, budget spent/progress/over-budget edge cases, and workspace access.
- `src/features/transactions/categorySuggestions.test.ts`: keyword matching, category-type compatibility, fallback behavior, and absent matching categories.
- `src/features/transactions/statementParser.test.ts`: date and amount parsing, descriptions, debit/credit inference and multiple statement formats.
- `src/test/setup.ts`: shared test setup. Vite config runs tests in jsdom with globals.

Backend tests (pytest)

`backend/tests/test_api.py` uses FastAPI's `TestClient`, mocked Supabase clients, environment monkeypatching, and dependency overrides. It checks health/auth requirements; token propagation; import input and duplicate fingerprints; category suggestion/backfill; organization RPC use; admin permission boundaries; and support-ticket authorization/not-found behavior.

Project commands

Command	What it does
`npm run dev`	Runs the Vite frontend and FastAPI service together.
`npm run dev:frontend` + `npm run api:dev`	Runs each service separately.
`npm run build`	TypeScript project build/check followed by Vite production bundle.
`npm run test`, `npm run test:api`	Vitest frontend suite and pytest API suite.
`npm run lint`	Runs Oxlint.
`npm run api:install`	Installs Python requirements into the active Python environment.

Presentation and project configuration

`src/index.css` defines root typography, colors/custom properties, global box sizing, page background, and base element defaults. The much larger `src/App.css` holds component/page layouts, responsive breakpoints, tables, dialogs, charts, status styles, landing page and account/admin/support screens. Styling uses semantic class names applied by JSX; responsive rules adapt the shell and grids for narrower screens.

`index.html` supplies document metadata and the root mount node. `vite.config.ts` enables the React plugin, Vitest/jsdom, setup file, and local API proxy. `tsconfig.app.json` enables modern ES/DOM types, bundler resolution, React JSX transform, no emit, and strict unused-local/parameter checks. Root and node TypeScript configs cover project references and Vite config.

`package.json` declares scripts plus React, Router, Supabase JS, date-fns, Recharts, Lucide, PDF.js and test/build tooling. `requirements.txt` lists FastAPI, Uvicorn, Supabase Python client, dotenv, HTTPX and pytest.

Environment and secrets

- Frontend uses `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY`; these are public/browser configuration values.
- Backend uses `SUPABASE_URL` and `SUPABASE_ANON_KEY`, plus optional `CORS_ORIGINS`.
- `SUPABASE_SERVICE_ROLE_KEY` is a server-only privileged secret for the invitation flow. Never place it in `VITE_*` variables or browser code.
- With no real browser Supabase settings, the UI selects demo access. Authentication and cloud persistence require Supabase configuration and applied schema/migrations.

README setup installs Node and Python, copies `.env.example`, installs npm/Python dependencies, runs the base SQL schema, configures email confirmation/redirects, applies relevant upgrades, and starts the app. The API docs are exposed by FastAPI at `/docs` while running.

13 · FILE MAP

What each project area contains

Path / group	Contents and role
`README.md`	Features, Windows setup, Supabase configuration, migrations and scripts.
`index.html`; `vite.config.ts`; `tsconfig*.json`; `package.json`; `requirements.txt`	HTML entry, dev/test/build configuration, TypeScript compiler settings and dependencies.
`scripts/dev.mjs`	Launches/stops frontend and API together.
`src/main.tsx`, `src/App.tsx`	React mount and app shell/routing/state/actions/page composition.
`src/types.ts`; `src/data/demo.ts`	Shared domain types and fictional local sample data/id/date helpers.
`src/lib/api.ts`; `src/lib/supabase.ts`	Authenticated typed HTTP client, data mappers, and optional browser Supabase client.
`src/utils/finance.ts`, `finance.test.ts`	Currency, reporting, filtering, budget and workspace calculations plus unit tests.
`src/features/auth/`	Supabase sign-in/signup/password recovery screen.
`src/features/account/`	Profile details and personal financial charts.
`src/features/organizations/`	Shared-workspace creation/list.
`src/features/admin/`	Organization overview, invitations, membership and roles.
`src/features/support/`	Support inbox/conversations with remote and demo behavior.
`src/features/transactions/StatementImportDialog.tsx`, `statementParser.ts`, `categorySuggestions.ts`	PDF text extraction/review, line parser, and category matching.
`src/features/transactions/*.test.ts`	Parser and categorization tests.
`src/test/setup.ts`	Shared frontend test initialization.
`src/index.css`, `src/App.css`	Global styling and responsive component/application styles.
`src/assets/`, `public/`	Bundled React/Vite/hero assets and public icons/branding media.
`backend/main.py`	FastAPI app, payload models, financial routes, import, organization/profile endpoints.
`backend/security.py`	Bearer-token authentication and workspace/platform admin authorization helpers.
`backend/categorization.py`	Server-side transaction typing and category rules.

Path / group	Contents and role
`backend/admin.py`, `backend/support.py`	Member administration and support-ticket route groups.
`backend/tests/test_api.py`	API, auth, import, categorization, authorization, and support tests.
`supabase/schema.sql`	Tables, constraints, indexes, helper functions, triggers, and RLS policies.
`supabase/seed_default_categories.sql`; `supabase/migrations/`	Default category seed and incremental database upgrades documented in chapter 10.

> **Reading strategy:** Begin with `src/types.ts`, then trace `src/main.tsx → src/App.tsx → src/lib/api.ts → backend/main.py → backend/security.py → supabase/schema.sql`. Read a feature component and its utility/tests together when exploring a single behavior.

Prepared from the project files in the workspace. The guide summarizes how the code is structured and operates; the source files remain authoritative for exact implementation details and current deployed database state.